Cinema Audience Forecasting using Time-Series Data

ABSTRACT

This project presents a comprehensive approach to forecasting cinema audience attendance by leveraging time-series data from two distinct platforms: BookNow, an online booking system, and CinePOS, an in-theatre point-of-sale system. The work encompasses data integration from eight related datasets, extensive preprocessing including missing value imputation and feature engineering, and the training of multiple gradient-boosted regression models. The final selected model, XGBoost, achieved a test R-squared score of 0.5556 and a mean squared error of 424.58, outperforming both Gradient Boosting Regressor and LightGBM on held-out validation data. The solution employs lag features, custom target encoding, and randomized hyperparameter search to optimize prediction accuracy for a 53-day forecast horizon.

INTRODUCTION

1.1 Problem Statement

Accurately predicting cinema audience attendance is a critical challenge for the entertainment and cinema management industry. Theatre operators must forecast daily visitor counts to optimize staffing schedules, resource allocation, concession inventory, marketing campaigns, and overall operational planning. Inaccurate forecasts can result in either wasteful over-provisioning or poor customer experiences due to under-preparation.

This project addresses the Cinema Audience Forecasting Challenge, which requires participants to predict daily audience counts for 826 theatres over a 53-day future window (March 1, 2024, through April 22, 2024) using historical data spanning January 1, 2023, through February 28, 2024.

1.2 Importance and Motivation

Cinema attendance is a complex, multi-factor phenomenon influenced by temporal patterns (weekdays versus weekends), calendar effects (holidays, festivals), theatre characteristics (type, location, area), and booking behavior across both online and offline channels. Understanding and modeling these interdependencies is essential for generating reliable forecasts.

Traditional heuristic approaches, such as simple historical averages, fail to capture the non-linear relationships and interaction effects between these diverse factors. Machine learning models, particularly ensemble tree-based methods, offer the ability to learn these complex patterns from data, making them well-suited for this forecasting task.

1.3 Real-World Applications

The outcomes of this project have direct applicability in several areas:

- Revenue management: Optimizing ticket pricing based on expected demand.
- Operational planning: Scheduling staff, ordering supplies, and managing theatre screens.
- Marketing strategy: Targeting promotional campaigns during predicted low-attendance periods.
- Strategic decision-making: Informing decisions about theatre expansion, renovation, or closure.
- Industry benchmarking: Enabling performance comparisons across theatres and regions.

2. METHODOLOGY / WORKING

2.1 Data Sources and Structure

The project utilizes eight interrelated datasets provided as part of the challenge:

- cinePOS_theaters.csv: Information about 4,690 CinePOS theatres, including theatre type, area, and geographic coordinates (latitude, longitude).
- booknow_theaters.csv: Information about 829 BookNow theatres with similar attributes.
- movie_theater_id_relation.csv: A mapping table linking 150 BookNow theatre IDs to their corresponding CinePOS theatre IDs.
- cinePOS_booking.csv: Over 1.6 million CinePOS booking records containing theatre ID, show datetime, booking datetime, and tickets sold.
- booknow_booking.csv: Approximately 68,336 BookNow booking records with theatre ID, show datetime, booking datetime, and tickets booked.
- booknow_visits.csv: Daily audience counts for BookNow theatres (214,046 records), which serves as the primary target variable.
- date_info.csv: Calendar information mapping dates to days of the week (547 entries).

- sample_submission.csv: The format and structure for the final forecast submission (38,062 entries for the prediction window).

The training data spans from January 1, 2023, to February 28, 2024, and the forecast period covers March 1, 2024, to April 22, 2024.

2.2 Data Preprocessing

The preprocessing phase involved several critical steps:

1. Datetime Conversion:
   - Converted show_datetime and booking_datetime columns in both booking datasets from string to date-only format.
   - Converted show_date in the visits dataset to proper date format.
2. Aggregation:
   - Grouped BookNow booking data by theatre ID and show date, summing the tickets_booked column.
   - Grouped CinePOS booking data by theatre ID and show date, summing the tickets_sold column.
3. Data Integration:
   - Merged BookNow visits with BookNow bookings on theatre ID and date.
   - Merged with BookNow theatre information for type, area, and location.
   - Merged with the theatre ID relation table to link BookNow to CinePOS theatres.
   - Merged with CinePOS booking aggregates and CinePOS theatre information.

   This produced a unified dataset of 214,046 records with 16 columns.

4. Missing Value Handling:
   - Theater type and area: First filled using the corresponding CinePOS values (from the merged data), then by the most frequent value (mode) for each theatre ID, and finally by the global mode.
   - Tickets booked and tickets sold: Filled using the median value for the corresponding show date, then by the overall median.
   - Latitude and longitude: Filled using the mode for each theatre, then the global mode.
   - String-to-numeric conversion: Extracted numeric IDs from book_theater_id and theater_area identifiers.
5. Redundant Column Removal:
   - Dropped duplicate columns resulting from the multi-table merge (e.g., show_datetime_x, show_datetime_y, theater_type_y, theater_area_y, latitude_y, longitude_y, cine_theater_id).

2.3 Feature Engineering

The following features were engineered from the date and theatre information:

1. **Temporal Features:**
   - is_weekday: Binary indicator (1 for Monday through Friday, 0 for Saturday and Sunday).
   - year: Year extracted from the show date (2023 or 2024).
   - month: Month number (1 through 12).
   - date: Day of the month.
   - daynum: Day of the week (0 = Monday through 6 = Sunday).
   - week: ISO week number.
2. **Lag Features (Time-Series Components):**
   - lag_1: Audience count from the previous day for the same theatre.
   - lag_2: Audience count from two days prior for the same theatre.
   - lag_7: Audience count from seven days prior for the same theatre (same day of the previous week).

   These lag features were generated by concatenating the training and submission datasets, sorting by theatre ID and date, and then applying grouped shifts. Missing lag values were imputed using a hierarchical strategy: first using the mean audience count by theatre ID, then by theatre area, and finally the global mean.

3. **Custom Target Encoding:**
   - A custom encoder class (CustomEncoder) was developed, inheriting from scikit-learn's BaseEstimator and TransformerMixin.
   - The encoder maps the book_theater_id to the mean audience count from the training data, with a fallback mapping using theater_area_x, and a final fallback to the global mean.
   - This target encoding allows the model to leverage theatre-specific historical attendance patterns as a numerical feature.
4. **Categorical Encoding:**
   - The theater_type_x column (with values such as Drama, Action, Other) was one-hot encoded using scikit-learn's OneHotEncoder with handle_unknown='ignore' to gracefully handle unseen categories during prediction.

2.4 Model Architecture

The final model feature set consisted of 13 features: is_weekday, year, month, date, daynum, latitude_x, longitude_x, book_theater_id, theater_area_x, theater_type_x, lag_1, lag_2, and lag_7.

Three individual models were trained and evaluated, each embedded in a scikit-learn Pipeline:

a) Gradient Boosting Regressor (GBR)

Pipeline: CustomEncoder followed by ColumnTransformer (one-hot encoding) followed by GradientBoostingRegressor.

Best hyperparameters (via RandomizedSearchCV with 50 iterations, 5-fold cross-validation):

- n_estimators: 100
- learning_rate: 0.05
- max_depth: 3
- subsample: 1.0
- min_samples_split: 2
- min_samples_leaf: 4
- max_features: log2

b) XGBoost Regressor (XGB)

Pipeline: CustomEncoder followed by ColumnTransformer (one-hot encoding) followed by XGBRegressor.

Best hyperparameters (via RandomizedSearchCV with 50 iterations, 5-fold cross-validation):

- n_estimators: 350
- learning_rate: 0.01
- max_depth: 10
- subsample: 0.6
- colsample_bytree: 0.7
- gamma: 0.05

- reg_alpha: 0.1
- reg_lambda: 1.5

c) LightGBM Regressor (LGBM)

Pipeline: CustomEncoder followed by ColumnTransformer (one-hot encoding) followed by LGBMRegressor.

Best hyperparameters (via RandomizedSearchCV with 50 iterations, 5-fold cross-validation):

- n_estimators: 200
- learning_rate: 0.05
- max_depth: 10
- num_leaves: 31
- min_child_samples: 20
- subsample: 0.8
- colsample_bytree: 0.8
- reg_alpha: 0
- reg_lambda: 1

d) Voting Regressor (Ensemble)

A VotingRegressor was also constructed by combining all three trained pipelines (GBR, XGB, LGBM) to produce averaged predictions.

2.5 Training Process

The dataset was split temporally:

- Training set: All records with show_date on or before January 31, 2024, and audience_count greater than 0.
- Validation/test set: All records with show_date after January 31, 2024, and audience_count greater than 0.

Hyperparameter optimization was performed using RandomizedSearchCV with:

- 50 random parameter combinations sampled from the defined search space.
- 5-fold cross-validation.

- Negative mean squared error as the scoring metric.
- Parallel execution (n_jobs=-1) for computational efficiency.

After selecting the best hyperparameters, each model pipeline was refitted on the complete dataset (all records with audience_count greater than 0) before generating final predictions.

For the final prediction phase, the best-performing model (XGBoost) was selected automatically based on R-squared scores. Predictions for the submission period were generated iteratively; for each record in the forecast window where audience_count was 0, the lag features were updated using previously predicted values to support recursive multi-step forecasting.

2.6 Evaluation Metrics

Two evaluation metrics were used: - R-squared (R2 Score): Measures the proportion of variance in the target variable explained by the model. A value closer to 1 indicates a better fit. - Mean Squared Error (MSE): Measures the average squared difference between predicted and actual values. Lower values indicate more accurate predictions.

RESULTS / OUTPUT

3.1 Model Performance Comparison

The following table summarizes the performance of all four models on the held-out test set (data after January 31, 2024):

Model	R2 Score	MSE
XGBoost	0.5556	424.58
Voting Regressor	0.5319	447.23
LightGBM	0.5273	451.59
Gradient Boosting	0.5111	467.05

XGBoost delivered the best performance with an R-squared score of 0.5556 and the lowest MSE of 424.58 among all individual and ensemble models.

3.2 Refit Performance on Complete Data

After refitting on the entire available dataset: - Gradient Boosting: R2 = 0.4958, MSE = 543.57 - XGBoost: R2 = 0.6031, MSE = 427.92 - LightGBM: R2 = 0.5364, MSE = 499.82

The XGBoost model remained the strongest performer, achieving an R-squared of 0.6031 after refitting, demonstrating that additional training data improved its predictive capability.

3.3 Key Observations

- Lag features (particularly lag_1 and lag_7) proved to be among the most informative predictors, reflecting the strong autocorrelation structure in daily cinema attendance data.
- The custom target encoding of theatre IDs effectively captured theatre-specific attendance levels, significantly improving model performance over one-hot encoding alone.
- Weekend versus weekday distinction (is_weekday) was a critical feature, consistent with the expected pattern of higher attendance on weekends.
- Temporal features such as month, day of month, and day of week captured seasonal and weekly cyclical patterns.
- The Voting Regressor ensemble did not outperform the best individual model (XGBoost), suggesting that XGBoost already captured the relevant patterns effectively.
- The XGBoost model showed a preference for a low learning rate (0.01) with a relatively high number of estimators (350) and moderate tree depth (10), indicating it benefited from slow, careful learning with moderately complex trees.

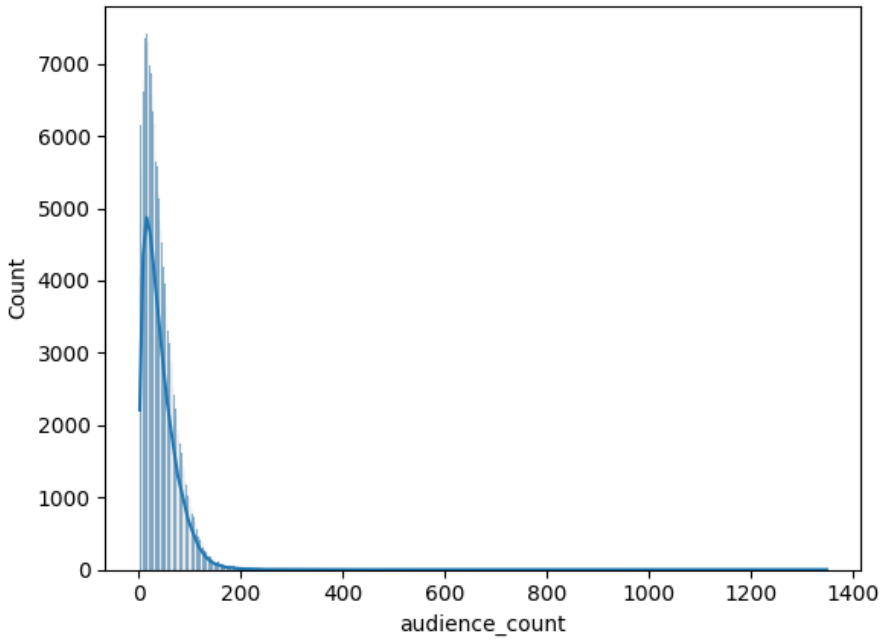

Fig 1: Audience Count

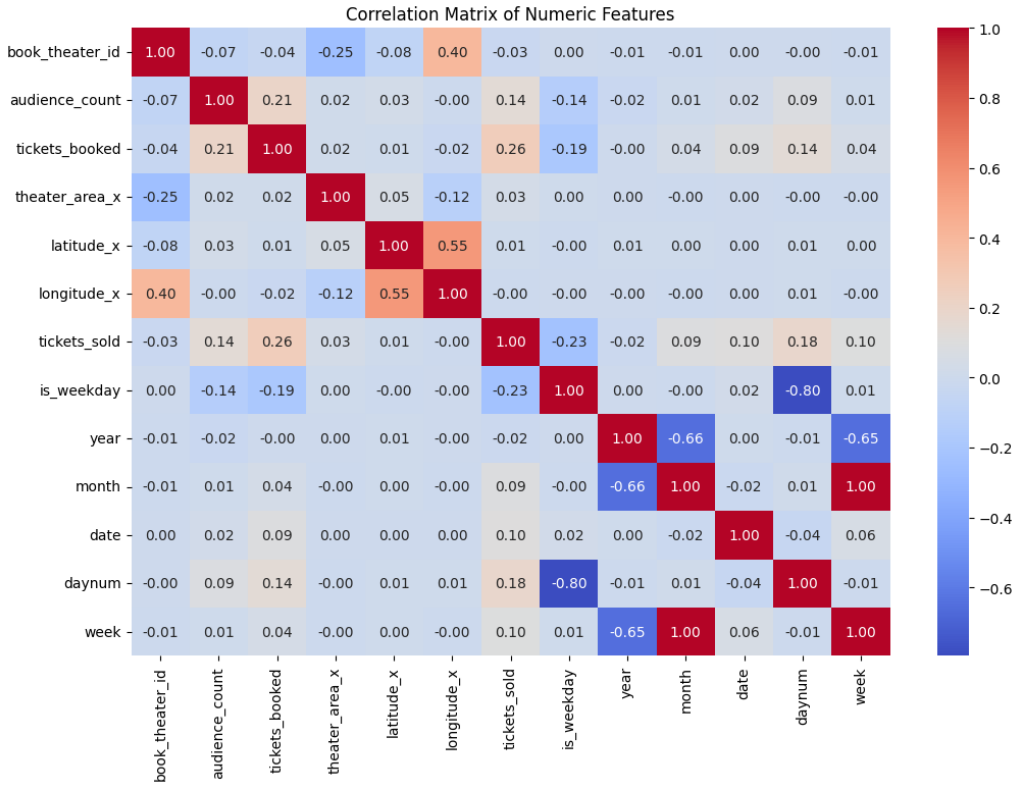

Fig 2: Correlation Heatmap

CONCLUSION

This project successfully developed a machine learning pipeline for forecasting cinema audience attendance using historical time-series data from both online (BookNow) and offline (CinePOS) platforms. Through systematic data integration, feature engineering, and model selection, the XGBoost regressor was identified as the best-performing model, achieving an R-squared score of 0.5556 on validation data and 0.6031 on the complete dataset after refitting. The use of lag features, custom target encoding, and randomized hyperparameter tuning contributed meaningfully to model accuracy. The methodology demonstrates that gradient-boosted ensemble methods can effectively capture the temporal, spatial, and categorical determinants of cinema attendance.

FUTURE SCOPE

Future work could explore deep learning architectures such as LSTM or Transformer-based models to better capture long-range temporal dependencies, incorporate external features like movie release schedules and holiday calendars, and investigate advanced ensemble stacking techniques for further performance improvement.